BAIL security

MEV-X
Differential

FINAL REPORT

May ‘2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	MevX – Differential - Audit Report
Website	mev-x.com
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/mev-x-project/MEV-X-Homelander/blob/7213522b54478535b1803bfba92650148ea53c90/contracts/HomelanderAlgebraPlugin.sol
Resolution 1	https://github.com/mev-x-project/MEV-X-Homelander/blob/36ace1f3ae13548c4159796b74ba91c766bbe4f8/contracts/HomelanderAlgebraPlugin.sol

2. Detection Overview

Please also refer to the previous audit report for already acknowledged issues.

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High						
Medium	2	2				
Low	12					
Informational	11	1		12		
Governance				10		
Total	25	3		22		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

HomelanderAlgebraPlugin

The HomelanderAlgebraPlugin is an afterSwap plugin which captures MEV profits instead of MEV bots. This audit is only a differential audit based on the following diff:

https://www.diffchecker.com/OyTdAYTP/

The following changes have been identified:

- renaming from MevXPlugin to HomelanderAlgebraPlugin
- incorporation of BEFORE_SWAP_FLAG and DYNAMIC_FEE flags
- incorporation of gas-related safety measures
- incorporation of beforeSwap logic for pluginFee and overrideFee
- refactoring of afterSwap control-flow
- incorporation of handlePluginFee logic

Appendix: beforeSwap

HomelanderAlgebraPlugin.beforeSwap() is the pre-swap hook used to supply Algebra with a dynamic fee override and an optional plugin fee. It is only consumed by Algebra when the pool's BEFORE_SWAP_FLAG is enabled and the pool caller is not the plugin itself.

The hook is reached through two Algebra swap paths:

AlgebraPool.swap(...)

→ _beforeSwap(...)

→ HomelanderAlgebraPlugin.beforeSwap(...)

And

AlgebraPool.swapWithPaymentInAdvance(...)

→ pull input token first

→ update reserves for actual amount received

→ unlock

→ _beforeSwap(...)

→ HomelanderAlgebraPlugin.beforeSwap(...)

It enables three distinct control-flows:

a: Caller is mevxExecutor; overrideFee is set to 1 and no pluginFee

b: Normal caller and !mevProtectionEnabled; pool's fee is used for swaps

c: Normal caller and mevProtectionEnabled, defaultFee is used as overrideFee and pluginFee is fetched via staticcall to router

Appendix: afterSwap

HomelanderAlgebraPlugin.afterSwap() is the in-scope post-swap hook used to trigger MEV-X arbitrage after an Algebra swap has already been calculated, token transfers/callbacks have occurred, reserves have been updated, and Algebra has unlocked the pool.

Algebra's relevant ordering is:

AlgebraPool.swap / swapWithPaymentInAdvance
→ _calculateSwap(...)
→ token transfers and callback/payment
→ _changeReserves(...)
→ emit Swap(...)
→ _unlock()
→ _afterSwap(...)
→ HomelanderAlgebraPlugin.afterSwap(...)

It essentially routes a call to self.runArbitrage and then checks for arbitrage possibility, crafts the route and executes the arbitrage call. If there is no arbitrage possibility or if the sender is the mevxExecutor, no route will be crafted and no arbitrage will be executed.

Appendix: handlePluginFee

HomelanderAlgebraPlugin.handlePluginFee() is the final in-scope endpoint for Algebra plugin-fee settlement.

Plugin fees are created during swaps when beforeSwap() returns a nonzero pluginFee. Algebra calculates the plugin-fee amount from swap fees, accrues it and later transfers those amounts to the current pool plugin through Algebra's delayed fee-transfer machinery.

The relevant Algebra flow is:

AlgebraPool._changeReserves(...)

→ _accrueAndTransferFees(..., receiverSlot = plugin.slot, feePendingSlot = pluginFeePending0.slot)

→ _transferFees(...)

→ IAlgebraPlugin(plugin).handlePluginFee(feeSent0, feeSent1)

The plugin then forwards the received tokens to profitDistributor and calls distributor logic.

Privileged Functions

- setPluginConfigToPool
- setConfigId
- setProfitDistributor
- setMevxExecutor
- setMevxRouter
- setMinGasLeft
- setCallGasBudget
- setDefaultFee
- setAuthorizedHandlePluginFeeCaller
- setMevProtectionEnabled

Core Invariants:

INV 1: If the sender is the mevxExecutor, no pluginFee should be taken

INV 2: If the sender is the mevxExecutor, no arbitrage should be executed

INV 3: The router must be robust against any side-effects from direct calls of afterInitialize and afterSwap hook executions

INV 4: If mevExecutor is the swapper, it should only take a fee of 1 (overrideFee)

INV 5: If !mevProtectionFeeEnabled, overrideFee and pluginFee must be zero and the globalState.lastFee should be considered as swap fee

INV 6: If swapper is not the mevExecutor and mevProtectionFeeEnabled, overrideFee becomes defaultFee and pluginFee is determined by low-level call return value.

INV 7: defaultFee can only ever be a maximum of 5%

INV 8: pluginFee can only ever be a maximum of 5%

Issue_01	Multi-hop swaps will donate fees to router / pool
Severity	**Medium**
Description	HomelanderAlgebraPlugin.afterSwap() receives the recipient argument from Algebra's pool-level swap hook. Homelander forwards this value through _afterSwap() into runArbitrage(), and after successful executor execution it calls profitDistributor.distributeProfit(configId, profitToken, recipient). In direct single-hop swaps, the pool-level recipient can be the user or the intended output recipient. In router-mediated multi-hop swaps, however, the pool-level recipient is often an intermediate contract. For exact-input multi-hop swaps, intermediate outputs are commonly sent to the router itself so that the router can use those tokens for the next hop. For exact-output nested swaps, an upstream pool can send output to a downstream pool that is still waiting for payment. Therefore, the pool-level recipient is not a reliable representation of the actual swap user. The root cause is a semantic mismatch between Algebra's per-hop recipient and Homelander's use of that recipient as a profit-distribution beneficiary. Algebra's hook supplies the recipient for the specific pool swap, not the end user of the entire route. Homelander forwards this value as the third argument to ProfitDistributor.distributeProfit(). The IProfitDistributor interface includes recipient-specific distribution semantics through the swapRecipient argument. If the distributor assigns a nonzero share to that argument, then multi-hop routes can credit the router or a pool rather than the real user.
Recommendations	Consider using tx.origin as a parameter. While that is no ideal solution and may have other side-effects, it appears to be the only partially satisfactory (on-chain) solution at this point.
Comments / Resolution	Resolved, the recipient parameter has been removed and as such there will be no profit distribution.

	The mevX team added the following comment: “There is no reliable way to recover the real swap originator inside afterSwap: the pool-level recipient points to an intermediate router for multi-hop swaps, and tx.origin resolves to the bundler's EOA in ERC-4337 account-abstraction transactions. We therefore removed the swapRecipient argument from HomelanderAlgebraPlugin's call to distributeProfit. Profit distribution will happen only among configured actors”

Issue_02	Lack of getCurrentFee function produces interface mismatch
Severity	**Medium**
Description	HomelanderAlgebraPlugin.defaultPluginConfig enables DYNAMIC_FEE together with BEFORE_SWAP_FLAG, AFTER_SWAP_FLAG, and AFTER_INIT_FLAG. The dynamic fee flag is necessary for Algebra to accept nonzero overrideFee and pluginFee values returned by Homelander's beforeSwap() hook. Algebra's swap execution uses beforeSwap() to obtain the executable fee values. However, Algebra's public fee view surface behaves differently. When dynamic fee mode is active, Algebra's fee() function queries the plugin through the dynamic-fee interface and expects the plugin to expose getCurrentFee(). Homelander does not implement getCurrentFee(). The root cause is an interface mismatch between the dynamic-fee flag Homelander enables and the view interface Algebra expects from dynamic-fee plugins. Homelander supplies executable fee data through beforeSwap(), but it does not supply the corresponding dynamic-fee view function used by Algebra's fee() getter. As a result, when an external caller invokes AlgebraPool.fee() on a Homelander-configured pool, Algebra detects that DYNAMIC_FEE is active and calls getCurrentFee() on the plugin. Since Homelander does not implement this selector, the view call can revert or fail. This does not break swaps because the swap path obtains fees from beforeSwap(). The affected surface is integration compatibility, fee discovery, quoting, monitoring, and any external system that relies on AlgebraPool.fee().
Recommendations	Consider implementing a getCurrentFee function which returns the defaultFee + pluginFee with NATSPEC that indicates that this might change based on the actual swap execution.

Comments / Resolution	Resolved, such a function has been added. It is important to note that the actual fee may differ depending on many different factors, for example the pluginFee could be different or the mevProtectionFeeEnabled can be turned to false.

Issue_03	Unguarded afterSwap hook allows for stealing idle funds in profitDistributor
Severity	**Low**
Description	In other issues we have already raised potential consequences of unguarded hooks. We were now successfully able to escalate this issue with the afterSwap hook in combination with stale/idle funds in the profitDistributor where the token transfer was successful but the distribution reverted via try/catch. If such a state is reached, anyone can simply trigger the profit distribution with their own address as recipient and the goal to catch the profitToken. This issue has only been raised as low severity because the external call to the mevxRouter determines the profit token.
Recommendations	Consider guarding the afterSwap function.
Comments / Resolution	Acknowledged. Team comment: “The ProfitDistributor is not expected to hold undistributed funds between executions — each successful arbitrage routes its profit straight through distributeProfit in the same transaction, so the leftover-funds situation this attack relies on should not occur during normal operation. Guarding afterSwap would add overhead to a hook that runs on every pool swap. We deliberately keep this hook unguarded.”

Issue_04	Delayed Algebra fee-flush can make gas estimation inaccurate
Severity	Low
Description	Algebra batches community fees and plugin fees in pending storage before transferring them. Homelander plugin fees are accumulated in pluginFeePending0/1, and community fees are accumulated in communityFeePending0/1. When Algebra later enters _changeReserves() with a nonzero current community or plugin fee, it can transfer historical pending fees and call the current plugin's handlePluginFee(). For Homelander, this means a normal swap can unexpectedly execute the handlePluginFee() control-flow before the post-swap afterSwap() hook. handlePluginFee() can transfer token0 and/or token1 to profitDistributor and then call profitDistributor.distributeProfit(). Only after this fee-settlement work does Algebra unlock the pool and call HomelanderAlgebraPlugin.afterSwap(), where _afterSwap() checks gasleft() >= minGasLeft for non-executor swaps. This ordering matters because a transaction's gas estimate can be accurate at the time it is simulated, but stale by the time it executes if the delayed fee-transfer state changes while the transaction is pending. The root cause is that Homelander's post-swap gas admission check is evaluated after Algebra's delayed fee-transfer machinery, while that fee-transfer machinery may or may not run depending on time, pending fee state, and prior transactions. A user or router can simulate a swap at a time when no fee flush occurs. The simulation accurately observes the current pool state and returns a gas estimate. Before the transaction executes, the Algebra fee-transfer interval may elapse, another transaction may increase pending fees, or another operation may create the first nonzero fee event that triggers the shared fee-transfer branch. When the user's transaction later executes, it can now trigger historical fee settlement, causing Algebra to transfer pending fees

	and call Homelander handlePluginFee(). That additional execution consumes gas before _afterSwap() reaches the minGasLeft check. If the transaction was submitted with gas based on the earlier no-flush simulation, it can reach _afterSwap() with less gas than expected and revert due Insufficient gas for afterSwap hook. Existing logic does not distinguish malicious gas-griefing from legitimate underestimation caused by delayed fee-transfer state changes. Therefore, a legitimate swap can revert because a delayed fee flush became due between simulation and execution.
Recommendations	Since this issue can regularly happen and swaps will basically fully revert then, it is worth a consideration to remove the gas enforcement, as there is not much upside in having this. 99.999% of users will never submit a transaction with dedicated gas to make try/catch executions run out of gas.
Comments / Resolution	Acknowledged.

Issue_05	Router change can desynchronize state
Severity	Low
Description	HomelanderAlgebraPlugin is bound to one immutable Algebra pool. During the intended initialization lifecycle, afterInitialize() derives the poolId from the bound Algebra pool address and calls mevxRouter.initializePoolExternally() with the initial sqrtPriceX96. Later, the same router address is used by beforeSwap() to query the MEV protection fee and by runArbitrage() to perform initialArbCheck() and constructArbitrageRoute(). The router address is mutable through setMevxRouter(). This function is owner-controlled and emits an event, but it only changes the local mevxRouter storage variable. It does not replay the pool initialization call for the new router and does not verify that the new router knows the bound Algebra pool. The root cause is that router state is treated as replaceable, while router-side pool initialization is lifecycle-dependent and not automatically migrated. The only automatic router initialization path is afterInitialize(), which runs during the Algebra pool initialization hook. Once the Algebra pool is already initialized, changing mevxRouter later does not cause Algebra to call afterInitialize() again. After the router address is changed, all future fee and arbitrage logic uses the new router. If the new router requires prior initializePoolExternally() state for the same poolId, then beforeSwap() and runArbitrage() can operate against uninitialized, stale, or incompatible router-side state. Homelander mostly treats router failures as soft failures: fee lookup can degrade to zero plugin fee, arbitrage checks can return early, and route-construction failures can be swallowed through the post-swap best-effort path. Existing logic does not prevent this because setMevxRouter() does not validate nonzero/code presence in the in-scope Algebra plugin, does not validate pool registration, and does not bind the new

	router to the current executor.
Recommendations	Consider initializing the new router if it has been changed. It should also be kept in mind that anyone can do that with an arbitrary price parameter.
Comments / Resolution	Acknowledged.

Issue_06	JIT-attack concerns for pluginFee distributions
Severity	**Low**
Description	Whenever pluginFees are transferred to the profitDistributor, a distribute call happens atomically. These fees can be transferred to vaults which could increase value per share (ERC4626-donation). If that ever happens, it becomes vulnerable to JIT attacks.
Recommendations	Consider carefully evaluating all profit distribution paths and preparing actions to prevent such JIT attacks.
Comments / Resolution	Acknowledged.

Issue_07	Decoupling of fee transfer and distribution can result in edge-cases
Severity	Low
Description	Homelander plugin fees are not transferred immediately when users pay them. During swaps, HomelanderAlgebraPlugin.beforeSwap() may return a nonzero pluginFee, and Algebra converts part of the swap fee into pluginFeePending0 or pluginFeePending1. Algebra later transfers these accumulated pending fees to the current plugin after the fee-transfer interval or another transfer condition is met. When the transfer condition is reached, a normal user swap can cause Algebra to transfer the pending plugin fees to HomelanderAlgebraPlugin and call handlePluginFee(). Homelander then forwards the received token amount to the current profitDistributor and attempts to execute profitDistributor.distributeProfit(configId, token, address(0)). This flow is important because the user whose swap triggers handlePluginFee() controls the timing of when historical pending plugin fees are sent into the distributor boundary, even though those fees may have been accrued by earlier users and under earlier configuration. The root cause is that Homelander separates value transfer from distribution finalization. In handlePluginFee(), the token transfer to profitDistributor is mandatory and occurs before the call to distributeProfit(). The later distributeProfit() call is wrapped in try/catch, so if distribution fails, Homelander still returns the expected selector to Algebra. As a result, if the token transfer succeeds but distributeProfit() reverts, the tokens have already left Homelander and reached the distributor, while Algebra's pluginFeePending0/1 can be cleared. Homelander does not store a retry record, does not track the amount that failed to distribute, and does not distinguish whether the distributor now holds idle assets.

	This also creates timing sensitivity. Since user swaps can trigger the delayed fee flush, users can determine when ProfitDistributor logic is executed. If the distributor performs downstream vault deposits, donations, or share-value-changing operations, the timing of the user-triggered swap may become economically relevant. That vault-specific MEV component depends on the missing distributor/vault implementation and is not fully proven from the in-scope code alone. Furthermore, this can amplify attacks where users can abuse such an idle asset state in the profitDistributor, then execute swaps and benefit from the full idle asset flush for the profitToken (as the distribution call does not forward an amount parameter, it is expected that the full balance is being flushed). Essentially, fees can be distributed under wrong semantics.
Recommendations	A fix for this issue would be to enforce the profitDistributor call but this could result in reverts during swaps. Hence, it is recommended to acknowledge this issue and enforce attempts that such a situation never occurs.
Comments / Resolution	Acknowledged.

Issue_08	communityFee can be dusted via small overrideFee
Severity	Low
Description	HomelanderAlgebraPlugin.beforeSwap() provides Algebra with two fee values: overrideFee and pluginFee. For normal protected swaps, Homelander can return defaultFee plus a router-provided MEV protection fee. For swaps where Algebra reports sender == address(mevxExecutor), Homelander returns an executor-specific fee tuple with overrideFee = 1 and pluginFee = 0. Algebra incorporates these values into swap math. If overrideFee is nonzero, Algebra uses it as the effective base fee, adds any pluginFee, and then calculates step.feeAmount for each swap step. Algebra's communityFee is then calculated as a percentage of this step.feeAmount. It is not a separate fee on swap volume. Therefore, reducing the effective fee to 1 also reduces the community-fee base to the smallest representable fee amount. This matters because Algebra community fee is a contextual protocol fee path, while Homelander's executor branch can materially alter the fee basis from which that contextual fee is calculated. The root cause is that Homelander's executor branch collapses the entire Algebra effective fee to 1, and Algebra calculates community fee only from the resulting step.feeAmount. Even if Algebra community fee is configured at 100%, the community fee receives only 100% of a near-zero fee amount. This does not bypass the community-fee percentage itself. Algebra still applies the configured community-fee share. The bypass occurs at the fee-base level: the executor branch causes the swap to generate only a dust step.feeAmount, so the community-fee result is also dust. The issue is reachable whenever the pool hook receives sender == address(mevxExecutor). If the MEV-X executor calls Algebra pools directly, this path is naturally used by executor routes. If the

	configured mevxExecutor is also a public router or public swap-forwarding contract, ordinary users may be able to route swaps through that address and receive the executor fee path. Existing logic does not prevent this because the executor branch is evaluated before mevProtectionFeeEnabled, before router fee lookup, and before any other fee path.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_09	MAX_MIN_GAS_LEFT is not aligned with MAX_CALL_GAS_BUDGET
Severity	**Low**
Description	The MAX_MIN_GAS_LEFT value is 2_500_000 and MAX_CALL_GAS_BUDGET is 5_000_000. During the afterSwap call, it checks for minGasLeft and then forwards callGasBudget, whereas the settings can allow for minGasLeft < callGasBudget which could make the runArbitrage fail if there's not enough gasleft..
Recommendations	Consider aligning both values and making sure that callGasBudget is below minGasLeft. And even this does not fully solve the problem. Another idea could be to simply remove the gas enforcement because 99.999% of users will never submit a transaction with dedicated gas to make try/catch executions run out of gas.
Comments / Resolution	Acknowledged.

Issue_10	Change of settings can result in adjustments of fee distribution
Severity	Low
Description	Homelander creates plugin-fee accrual through beforeSwap(). When mevProtectionFeeEnabled is enabled and mevxRouter.getMevProtectionFee(defaultFee) returns a valid nonzero fee, Homelander returns an Algebra pluginFee. Algebra then charges the effective swap fee, calculates the plugin-fee share from the swap fee, and stores the resulting amount in pluginFeePending0 or pluginFeePending1. These amounts are not immediately transferred in all cases. Algebra batches community fees and plugin fees and transfers them only after Constants.FEE_TRANSFER_FREQUENCY, which is 8 hours, or if pending values exceed the packing threshold. Until then, the pending fee is a pool-level Algebra accounting value. When the delayed transfer finally occurs, Algebra transfers the pending plugin fees to the **current** pool plugin address and calls handlePluginFee() on that current plugin. If the current plugin is still HomelanderAlgebraPlugin, Homelander then forwards the received token to the **current** profitDistributor and calls distributeProfit() with the **current** configId. The root cause is that the fee creation point and fee settlement point are separated, but the relevant distribution metadata is not checkpointed at creation time. At the moment the fee is created, the economically relevant state includes the active Homelander plugin instance, configId, profitDistributor, and MEV protection configuration. However, Algebra only stores token amounts in pluginFeePending0 / pluginFeePending1. It does not bind those pending amounts to the Homelander configuration that produced them. At settlement time, Algebra uses the current plugin storage slot as the fee recipient. Homelander then uses its current profitDistributor

	and current configId to distribute the fee. Therefore, fees paid under one configuration can later be distributed under another configuration.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_11	Direct external calls to afterInitialize, beforeSwap and afterSwap
Severity	**Low**
Description	The contract exposes these three functions completely unguarded which means anyone can call these. Sensitive external calls to corresponding addresses are being executed which can have unexpected down-stream effects which are currently invisible.
Recommendations	Consider analyzing the impact of a direct call to these functions, including the use of unexpected parameters.
Comments / Resolution	Acknowledged.

Issue_12	Revert of swap due to blacklisted profitDistributor address for tokens
Severity	Low
Description	HomelanderAlgebraPlugin.beforeSwap() can return a nonzero pluginFee when mevProtectionFeeEnabled == true and mevxRouter.getMevProtectionFee(defaultFee) returns a valid plugin fee. Algebra consumes this value during swap calculation and accrues the resulting plugin-fee amount into pluginFeePending0 or pluginFeePending1. Algebra does not necessarily transfer these plugin fees immediately. Instead, it stores them in the pool and later flushes them through _changeReserves() once the fee-transfer interval has elapsed or a pending-fee threshold is reached. When the transfer condition is met, Algebra transfers the pending plugin fee to the current pool plugin and then calls IAlgebraPlugin(plugin).handlePluginFee(feeSent0, feeSent1). In the in-scope plugin, handlePluginFee() checks that the caller is either the bound algebraPool or an address enabled in authorizedHandlePluginFeeCallers. It then reads token0 and token1 from the bound Algebra pool and forwards the received fee amounts to profitDistributor using SafeERC20.safeTransfer. Only after this transfer does it attempt profitDistributor.distributeProfit(configId, token, address(0)), which is wrapped in try/catch. The critical distinction is that the token transfer to profitDistributor is hard-fail, while the later distributor call is soft-fail. The root cause is that handlePluginFee() assumes that profitDistributor is always able to receive the pool tokens. This assumption is not enforced by the plugin and may not hold for tokens with blacklist, sanctions-list, transfer-restriction, allowlist, or recipient-validation logic. If the pool token rejects transfers to the current profitDistributor,

	then IERC20(token).safeTransfer(address(profitDistributor), pluginFee) reverts. Since this revert occurs before the try/catch-protected distributeProfit() call, the entire handlePluginFee() call reverts. Algebra requires the plugin call to return IAlgebraPlugin.handlePluginFee.selector. If handlePluginFee() reverts, Algebra's _changeReserves() reverts, and the whole swap or flash operation that triggered the fee flush reverts. Because the transaction reverts atomically, the pending plugin-fee state remains in Algebra and the same revert condition can recur on later fee-transfer-triggering operations.
Recommendations	Consider changing the profitDistributor address if such an unlikely state ever happens.
Comments / Resolution	Acknowledged.

Issue_13	Pending pluginFees can remain unflushed
Severity	Low
Description	Algebra stores delayed fees in communityFeePending0, communityFeePending1, pluginFeePending0, and pluginFeePending1. The internal _accrueAndTransferFees() function supports two cases: adding new fees and potentially transferring them, or transferring already-pending fees when no new fees are added but the transfer interval has elapsed. However, _accrueAndTransferFees() is only reached through _changeReserves(). The relevant _changeReserves() branch is guarded by whether the current operation supplies nonzero communityFee0, communityFee1, pluginFee0, or pluginFee1. This means the delayed-transfer timestamp alone is not enough to flush existing Homelander plugin fees. The root cause is a mismatch between the delayed-transfer helper and its call gate. The helper can flush stale fees with zero new fee amounts, but the outer _changeReserves() function does not call the helper unless the current operation has a nonzero fee component. If a pool has existing pluginFeePending0/1, but later operations create no new community fees and no new plugin fees, the pending fees are not inspected or transferred, even if more than 8 hours have elapsed. This can occur in Homelander after protected swaps have accrued plugin fees, followed by a state where mevProtectionFeeEnabled is disabled, the MEV router returns zero or invalid plugin fees, Algebra community fee is zero, or the pool becomes inactive. The result is not just delayed settlement. It can become indefinite non-settlement until some later operation creates a nonzero current community fee or plugin fee.
Recommendations	Consider making sure to activate mevProtectionFeeEnabled in such

	a scenario.
Comments / Resolution	Acknowledged.

Issue_14	Lack of sync and skim exposure
Severity	Low
Description	Algebra pools expose sync() and skim() as permissioned maintenance functions. Both functions are callable only by the current pool plugin. sync() updates the pool's internal reserve accounting to match current token balances, while skim() removes excess token balances above tracked reserves to the plugin caller. These functions are relevant when token balances and tracked reserves diverge, such as after direct token transfers, non-standard token behavior, misrouted transfers, or recovery/migration states. When HomelanderAlgebraPlugin is installed as the Algebra pool plugin, it becomes the only address allowed to call these functions. However, the in-scope plugin only implements hook and fee-distribution logic. It does not expose any owner-controlled or operational function that calls algebraPool.sync() or algebraPool.skim(). The root cause is an architectural mismatch between Algebra's plugin permission model and Homelander's implemented functionality. Algebra assigns reserve-maintenance authority to the current plugin, but Homelander does not implement a path to exercise that authority. This does not affect normal swaps in the happy path, because Algebra also updates reserves through ordinary pool operations. However, in abnormal states, operators may need to intentionally synchronize reserves or remove excess balances without relying on a swap, mint, burn, flash, or other pool operation. If those ordinary operations are unavailable, unsafe, or reverting due fee-settlement side effects, the plugin-only maintenance functions cannot be used while Homelander remains the current plugin.
Recommendations	Consider implementing these functions and make them only-owner callable

Comments / Resolution	Acknowledged.

Issue_15	Lack of pluginFee if communityFee is set to 100%
Severity	**Informational**
Description	The plugin can determine the pluginFee but it will not have any effect if the communityFee within the AlgebraPool is set to 100%, as this would then deduct any fee and add it towards the communityFee, leaving no pluginFee left.
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged.

Issue_16	DoS of swaps under several edge-cases
Severity	Informational
Description	Please note that this issue is rather Algebra than plugin related and as such it is only marked as informational severity. Algebra uses delayed fee-transfer accounting for both community fees and plugin fees. Community fees are stored in communityFeePending0/1, while Homelander-created plugin fees are stored in pluginFeePending0/1. These pending amounts are transferred later through a shared _changeReserves() fee-settlement branch. Homelander participates in this flow because beforeSwap() can return a nonzero pluginFee, causing Algebra to calculate a plugin-fee amount and enter _changeReserves() with nonzero plugin-fee arguments. Once that branch is entered, Algebra processes community-fee settlement before plugin-fee settlement. Algebra also allows the community vault to be set to address(0). When this happens while community fees are already pending, the pending community-fee amounts remain in storage unless separately settled. A later Homelander plugin-fee swap can therefore awaken old community-fee state. The root cause is that delayed community-fee pending state is not checkpointed or cleared before the community vault can become address(0), and Algebra later settles pending community fees using the current communityVault. If stale communityFeePending0/1 exists and communityVault is now zero, any later operation that enters the shared fee-transfer branch can attempt to transfer those stale community fees to address(0). A Homelander plugin-fee swap is one such operation. Even if current communityFee is zero, a nonzero current pluginFee causes _changeReserves() to enter the fee-transfer branch. Algebra then attempts community-fee settlement first because stale community fees exist and the transfer condition is met. If the token rejects

	transfers to address(0), this transfer reverts. The revert occurs before Homelander plugin-fee settlement and reverts the whole swap. Existing logic does not prevent this because setting communityFee = 0 does not erase historical communityFeePending0/1, and Homelander cannot bypass Algebra's community-fee settlement order. A similar but already known issue exists if plugin is set to address(0) but there are stale pluginFees.
Recommendations	Consider acknowledging this edge-case.
Comments / Resolution	Acknowledged.

Issue_17	Aggregated swap fee can become uneconomical
Severity	Informational
Description	HomelanderAlgebraPlugin.beforeSwap() returns an Algebra overrideFee and pluginFee. The overrideFee is sourced from Homelander's defaultFee, which is capped at AlgebraConstants.MAX_DEFAULT_FEE = 50_000. When MEV protection fees are enabled, Homelander also asks the configured mevxRouter for a plugin fee and accepts it if it is no greater than the same MAX_DEFAULT_FEE. Algebra does not treat these two values as alternatives. If overrideFee is nonzero, Algebra calculates the effective swap fee as overrideFee + pluginFee. Therefore, two independently valid 5% components can combine into a 10% effective swap fee. The root cause is that Homelander validates the base fee and plugin fee independently but does not enforce an aggregate cap matching AlgebraConstants.MAX_DEFAULT_FEE. Algebra later combines the values into one effective swap fee and only rejects if that fee reaches the hard denominator limit of 1_000_000. This means that Homelander's apparent 5% cap is not a cap on the total user-facing swap fee. It is a cap on each component. If the owner configures defaultFee = 50_000 and the router returns pluginFee = 50_000, Algebra accepts both and charges 100_000, equal to 10%. This is not a direct swap DoS because Algebra considers the fee valid. Swaps can still execute if the user's amount and slippage tolerate the fee. The issue is that the protocol can enter a high-fee state that makes swaps economically unattractive or causes router-level slippage checks to fail.
Recommendations	Consider making sure that governance sets reasonable fees.
Comments / Resolution	Acknowledged.

Issue_18	handlePluginFee can be unexpectedly triggered via flashloan execution
Severity	**Informational**
Description	Whenever a flashloan is taken, this will invoke the _changeReserves function which then in turn will eventually trigger the handlePluginFee call. This is not an issue but rather a note to raise awareness of this behavior.
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged.

Issue_19	Direct afterSwap call can be used to trigger arbitrage without previous swap
Severity	Informational
Description	HomelanderAlgebraPlugin.afterSwap() is intended to be invoked by the bound Algebra pool after a swap has completed. In the genuine Algebra flow, the pool executes swap accounting, unlocks, and then calls the plugin's afterSwap() hook with the actual pool-level sender, recipient, direction, and swap deltas. Homelander forwards these values into _afterSwap(), derives the poolId from the bound Algebra pool, and then self-calls runArbitrage() with the supplied swap metadata. The plugin uses a self-call guard on runArbitrage(), so external callers cannot directly call runArbitrage(). However, afterSwap() itself is externally callable and does not verify that the caller is address(algebraPool). This makes afterSwap() a public wrapper around the protected runArbitrage() flow. The root cause is that Homelander authenticates runArbitrage() through msg.sender == address(this), but does not authenticate the hook function that creates the self-call. An arbitrary caller can invoke afterSwap() directly, provide arbitrary values for sender, recipient, zeroToOne, amount0, and amount1, and cause Homelander to call runArbitrage() on itself. Inside _afterSwap(), the pool identity is not taken from msg.sender. It is hardcoded to address(algebraPool). Therefore, a direct external caller can cause Homelander to ask the MEV-X router about the real bound Algebra pool while supplying synthetic swap metadata. This does not directly mutate Algebra pool state and does not create a real swap. The risk is that MevxRouter.constructArbitrageRoute() or an off-chain backend may treat calls originating from the Homelander plugin as evidence that a genuine post-swap hook occurred. If the router/backend

	trusts the plugin context or supplied deltas without independently validating real pool state, the caller can trigger route construction or execution attempts without a preceding pool swap.
Recommendations	Consider inspecting whether this can become a problem.
Comments / Resolution	Acknowledged.

Issue_20	Arbitrage can revert in multi-hops
Severity	**Informational**
Description	In Algebra exact-output multi-hop routing, the router begins from the final output pool and executes upstream payment swaps through nested callbacks. During this process, an outer pool can remain locked while an inner pool completes its swap and calls its afterSwap() hook. Homelander executes its post-swap logic in afterSwap(). The plugin calls runArbitrage() through a gas-capped self-call, and runArbitrage() may construct and execute a MEV-X route through mevxExecutor.executeRoute(). If the MEV-X route includes a pool that is still locked by the outer exact-output swap, Algebra's lock guard prevents that nested swap from executing. The root cause is hook timing relative to exact-output nested callbacks. Homelander's afterSwap() executes after an individual pool swap completes, not after the entire router-level multi-hop route completes. In exact-output routing, another pool from the same route can still be locked higher in the call stack. If MevxRouter.constructArbitrageRoute() builds a route that touches the still-locked pool, the executor route reverts when it tries to use that pool. Homelander catches executor failure, so the original user swap continues. Therefore, this is not a user-swap revert issue under the current in-scope control-flow. It is a MEV-X capture reliability issue.
Recommendations	Consider acknowledging this design limitation.
Comments / Resolution	Acknowledged.

Issue_21	Inconsistent handling of address(0) cases
Severity	Informational
Description	HomelanderAlgebraPlugin depends on three mutable external integrations: mevxRouter, mevxExecutor, and profitDistributor. These addresses are set during construction and are consumed by the core runtime paths of the plugin. The router is used for pool initialization, MEV protection fee lookup, initial arbitrage checks, and route construction. The executor is used for executing constructed arbitrage routes and is also used as the identity that receives special swap-fee and recursion-prevention treatment. The profit distributor is used as the recipient and distribution endpoint for both arbitrage profits and Algebra plugin fees. The constructor explicitly rejects zero values for these integration addresses. However, the corresponding owner-controlled setter functions do not preserve the same invariant after deployment. This means the live plugin can be moved into a state that would have been rejected during deployment. The root cause is an inconsistent validation boundary between deployment-time initialization and post-deployment configuration. The constructor treats zero integration addresses as invalid, but setMevxRouter(), setMevxExecutor(), and setProfitDistributor() directly overwrite storage without checking that the new address is nonzero or otherwise suitable. This is not only a cosmetic inconsistency. The affected addresses are consumed by live swap and fee-settlement flows. If mevxRouter is zero, low-level router calls can degrade into empty return data or ignored failures, causing fee lookup, pool initialization, and route construction to silently stop functioning. If mevxExecutor is zero or unsuitable, Homelander can construct routes but fail to execute them, while the original swap may still finalize due the plugin's try/catch and ignored low-level call behavior. If profitDistributor is zero or unsuitable, plugin-fee forwarding can revert, route profits can be misrouted, and distribution calls can become no-ops or

	failures depending on token and call behavior. Existing access control only restricts who can create the state. It does not prevent the invalid state from existing.
Recommendations	Consider enforcing consistent address(0) validation.
Comments / Resolution	Resolved.

Issue_22	Deployment execution can allow for hook-free swap window
Severity	**Informational**
Description	HomelanderAlgebraPlugin is intended to operate as an Algebra plugin using a default hook configuration that enables pre-swap fee control, post-swap arbitrage, post-initialization router registration, and dynamic fee mode. However, Algebra plugin installation and plugin-configuration activation are separate control-flows. The Algebra pool administrator installs the plugin address through the pool's plugin-setting flow. Algebra then resets the pool's pluginConfig to zero. Separately, the Homelander owner must call setPluginConfigToPool() to set Homelander's intended defaultPluginConfig. Until that second step occurs, the pool has the Homelander plugin address installed but no active Homelander hook flags. The root cause is that Homelander's deployment sequence is not atomic across Algebra plugin installation and hook activation. During the intermediate state, plugin points to Homelander but pluginConfig == 0. Algebra's hook dispatcher checks the bitmap before invoking hooks. With no flags active, Algebra does not call HomelanderAlgebraPlugin.beforeSwap() or afterSwap(). This means swaps during the deployment gap do not receive Homelander's defaultFee override, do not query the MEV-X router for plugin fees, do not accrue Homelander plugin fees, and do not trigger runArbitrage() after the swap. Algebra instead falls back to its normal globalState.lastFee behavior. Existing logic does not prevent public swaps during this window unless external operational controls prevent trading. The contract itself does not atomically install and activate the plugin.
Recommendations	Consider elaborating whether that exposes an issue. The deployment is currently treated under assumptions.

Comments / Resolution	Acknowledged.

Issue_23	callbackGasBudget is similarly used for different executions
Severity	Informational
Description	HomelanderAlgebraPlugin uses one gas budget variable, callGasBudget, for three materially different flows. During beforeSwap(), the plugin uses it for a staticcall to mevxRouter.getMevProtectionFee(defaultFee). During afterInitialize(), it uses it to call mevxRouter.initializePoolExternally(...). During _afterSwap(), it uses it as the gas cap for the self-call into runArbitrage(), which can perform router checks, route construction, executor execution, and profit distribution. These flows have different expected gas profiles. The post-swap arbitrage path can legitimately require a large gas envelope because it may construct and execute a route. The pre-swap fee getter should normally be a scalar lookup. The initialization call is a lifecycle call, not a per-swap arbitrage execution path. The plugin nevertheless applies the same default callGasBudget, which is initialized to MAX_CALL_GAS_BUDGET = 5_000_000. The root cause is that the plugin exposes a single gas budget for heterogeneous external-call responsibilities. A value that is reasonable for runArbitrage() can be excessive for the beforeSwap() fee lookup, because every protected user swap can expose up to that budget to the router's getMevProtectionFee() staticcall. If the router fee getter is state-heavy, faulty, or unexpectedly expensive, user swaps can pay substantial gas before Algebra swap execution begins. The same issue applies less frequently to afterInitialize(). A high gas budget may be acceptable during deployment, but using the arbitrage execution budget for initialization makes initialization gas behavior depend on the same parameter used for runtime arbitrage. If the initialization call consumes or requires unexpected gas, Algebra pool initialization can finalize while Homelander ignores the router call result.

	Existing logic does not distinguish these paths. setCallGasBudget() only applies a maximum bound, and the configured value is reused everywhere.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_24	Swaps may revert if mevxExecutor uses router for arbitrage
Severity	Informational
Description	Algebra's SwapRouter.exactOutput() uses a storage variable called amountInCached to store the final input amount during multi-hop exact-output execution. The router starts with the final output pool and executes upstream payment swaps through nested callbacks. The deepest callback eventually sets amountInCached, and after all swaps unwind, the top-level exactOutput() function reads this value, checks it against amountInMaximum, and resets it to the default sentinel. Homelander's afterSwap() runs after each individual pool swap, not after the entire router-level route completes. During this hook, Homelander can enter runArbitrage() and call mevxExecutor.executeRoute(). If the executor performs swaps by calling the same Algebra router, rather than directly calling pools, the executor can reenter the router while the original exact-output call is still active. The root cause is a storage-cache collision in Algebra's router when nested exact-output calls are made to the same router instance. The outer user route relies on amountInCached remaining intact between the deepest exact-output callback and the final read in exactOutput(). A nested executor route that calls the same router exact-output function can overwrite or reset that shared storage variable. If the nested executor exact-output call completes successfully, it resets amountInCached to the default sentinel. Control then returns to Homelander and eventually back to the original user's router call. The top-level exactOutput() reads amountInCached, now reset, and can fail the final amountIn <= amountInMaximum check. This is not prevented by Homelander's try/catch, because the nested executor call can succeed. The later revert occurs when the original router resumes and reads the corrupted cache. The issue is conditional on executor design. If the executor calls Algebra pools

	directly, or uses a different router instance, or does not use exact-output router functions, this cache collision does not occur. This issue is only raised as informational because the mevxExecutor is expected to directly call the plugin, as indicated by the msg.sender = sender usage in afterSwap, which would then otherwise not propagate the mevxExecutor = sender short circuit.
Recommendations	Consider making sure that the mevxExecutor never uses the router for arbitrage.
Comments / Resolution	Acknowledged.

Issue_25	Previously acknowledged issues
Severity	**Informational**
Description	Within the first report, there are previously acknowledged issues such as revert due to malformed return data and other issues. These need to be kept in mind for the new version as well.
Recommendations	Consider keeping these issues in mind.
Comments / Resolution	Acknowledged.